

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_c5395511-084\agent-aea940cf8cf35bb6e.jsonl`
- SHA-256: `b86cb6570ae9d0697552556b12a296675aadee87ffc2e88ca4673018b5ef826d`
- Source modified: `2026-07-06T21:30:18+00:00`
- Imported at: `2026-07-08T16:00:11+00:00`
- project: `wf\_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:29:24.640Z)

Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there, or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (db._apply_schema on aiosqlite ":memory:").

WHAT F2 DOES:

- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video / _content_item_to_video / get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences (json.loads(policy.config_json)); TopicPreferenceStage (Tier 2): exclude-veto, include any/all, casefold substring match over source.title + content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty); _content_item_preview(...) builds a throwaway ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn, account, source, item) loads the default policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now: matched=engine.evaluate(); if matched and source: run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a /set (F1's merge) and are read via the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate: ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium (edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /

pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity.

FINDING (stage_config): Malformed topics config fails closed and silently drops ALL matched deliveries with no surfaced reason

severity medium | src/tg_video_filter/policy.py:126

summary: Any parse/validation error in the topics config makes TopicPreferenceStage.evaluate return StageResult.fail, which the wiring turns into matched=False, so every otherwise-matching video is silently dropped until the config is fixed.

scenario: An operator (or a future /set-topics command) writes policies.config_json with a structurally invalid topics block, e.g. {"topics": {"include": "ai"}} (string instead of array) or {"topics": {"include": [123]}}. load_topic_preferences raises ValueError; the stage returns fail; on_new_message (telethon_client.py:291) sets matched=topic_result.passed=False; every matched video is dropped. Repro confirmed: case 4 -> 'invalid topic config: topic include/exclude values must be arrays', passed=False; case 7 -> 'topic include/exclude arrays must contain strings', passed=False. The only visible signal is the reason appended to the log line (telethon_client.py:290,322) ? there is no user-facing error and no distinction from a legitimate topic mismatch. Because the F2 diff makes NO command able to set topics safely-validated (only save_policy_config/manual edit exist), this is latent today, but the fail-closed choice means the first bad config edit blackholes all delivery.

evidence: policy.py:122-126 catches (JSONDecodeError, ValueError) and returns StageResult.fail(); telethon_client.py:289-291 does `matched = topic\_result.passed`. Repro output: `4 include=string -> DROPPED: False - invalid topic config: topic include/exclude values must be arrays`; `7 include=[123] -> DROPPED: False`; `13 config\_json array -> DROPPED: False`. ADR 0012 does not specify fail-open vs fail-closed for malformed config.

rec: Decide the failure posture explicitly. Fail-open (treat an unparseable topics block as 'topics disabled' and pass) is safer for a delivery pipeline ? a config typo should not stop all delivery. If fail-closed is intentional for safety, surface the invalidity loudly (dedicated log level/alert, and reject the config at write time in save_policy_config rather than silently at evaluate time) so it cannot silently blackhole delivery.

2. assistant (2026-07-06T21:29:27.046Z)

I'll verify this finding by reading the cited code and running a repro.

3. user (2026-07-06T21:29:28.075Z)

```

1      """Policy pipeline primitives for ADR 0012.
2
3      The policy layer is deliberately small and framework-free in F1: stages receive
4      an immutable context, return a structured result, and the evaluator stops on
5      the first failure. Later Track F phases add concrete stages.
6      """
7
8      from __future__ import annotations
9
10     import json
11     from dataclasses import dataclass, field
12     from typing import Protocol
13
14     from tg_video_filter.models import ContentItem, Policy, Source
15
16
17     @dataclass(frozen=True)
18     class PolicyContext:
19         """Immutable snapshot available to a policy stage."""
20
21         content_item: ContentItem
22         source: Source
23         policy: Policy
24         fetcher: object | None = None
25         budget: object | None = None
26
27
28     @dataclass(frozen=True)
29     class StageResult:
30         """Outcome of one policy stage."""

```

```

31
32     passed: bool
33     reason: str
34     stage_name: str
35     metadata: dict[str, object] = field(default_factory=dict)
36
37     @staticmethod
38     def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
39         return StageResult(
40             passed=True,
41             reason=reason,
42             stage_name=stage_name,
43             metadata=metadata,
44         )
45
46     @staticmethod
47     def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
48         return StageResult(
49             passed=False,
50             reason=reason,
51             stage_name=stage_name,
52             metadata=metadata,
53         )
54
55
56 class PolicyStage(Protocol):
57     """A single stateless gate in the policy pipeline."""
58
59     async def evaluate(self, ctx: PolicyContext) -> StageResult:
60         """Evaluate ``ctx`` and return a stage result."""
61         ...
62
63
64 @dataclass(frozen=True)
65 class TopicPreferences:
66     """Parsed ADR 0012 topic preference config."""
67
68     include: tuple[str, ...] = ()
69     exclude: tuple[str, ...] = ()
70     mode: str = "any"
71
72     @property
73     def enabled(self) -> bool:
74         return bool(self.include or self.exclude)
75
76
77 def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
78     """Parse ``topics`` from a raw policy config object."""
79     raw_topics = config.get("topics")
80     if raw_topics is None:
81         return TopicPreferences()
82     if not isinstance(raw_topics, dict):
83         raise ValueError("topics must be a JSON object")
84     mode = raw_topics.get("mode", "any")
85     if mode not in ("any", "all"):
86         raise ValueError("topics.mode must be 'any' or 'all'")
87     return TopicPreferences(
88         include=_string_tuple(raw_topics.get("include")),
89         exclude=_string_tuple(raw_topics.get("exclude")),
90         mode=str(mode),
91     )
92
93
94 def load_topic_preferences(policy: Policy) -> TopicPreferences:
95     """Load topic preferences from ``policy.config_json``."""

```

```

96     decoded = json.loads(policy.config_json)
97     if not isinstance(decoded, dict):
98         raise ValueError("policy config_json must decode to a JSON object")
99     return parse_topic_preferences(decoded)
100
101
102 def _string_tuple(value: object) -> tuple[str, ...]:
103     if value is None:
104         return ()
105     if not isinstance(value, list):
106         raise ValueError("topic include/exclude values must be arrays")
107     topics: list[str] = []
108     for item in value:
109         if not isinstance(item, str):
110             raise ValueError("topic include/exclude arrays must contain strings")
111         topic = item.strip()
112         if topic:
113             topics.append(topic)
114     return tuple(topics)
115
116
117 class TopicPreferenceStage:
118     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
119
120     stage_name = "topic-preferences"
121
122     async def evaluate(self, ctx: PolicyContext) -> StageResult:
123         try:
124             prefs = load_topic_preferences(ctx.policy)
125         except (json.JSONDecodeError, ValueError) as exc:
126             return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
127         if not prefs.enabled:
128             return StageResult.pass_(self.stage_name, "topic preferences disabled")
129
130         haystack = _topic_haystack(ctx)
131         matched_excludes = _matched_topics(prefs.exclude, haystack)
132         if matched_excludes:
133             return StageResult.fail(
134                 self.stage_name,
135                 f"excluded topic matched: {matched_excludes[0]}",
136                 matched_exclude_topics=list(matched_excludes),
137             )
138
139         if not prefs.include:
140             return StageResult.pass_(self.stage_name, "no include topics configured")
141
142         matched_includes = _matched_topics(prefs.include, haystack)
143         if prefs.mode == "any":
144             if matched_includes:
145                 return StageResult.pass_(
146                     self.stage_name,
147                     f"matched include topic: {matched_includes[0]}",
148                     matched_include_topics=list(matched_includes),
149                 )
150             return StageResult.fail(self.stage_name, "no include topics matched")
151
152         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
153         if missing:
154             return StageResult.fail(
155                 self.stage_name,
156                 f"missing required topic: {missing[0]}",
157                 matched_include_topics=list(matched_includes),
158                 missing_include_topics=list(missing),
159             )

```

```

160         return StageResult.pass_(
161             self.stage_name,
162             "matched all include topics",
163             matched_include_topics=list(matched_includes),
164         )
165
166
167     def _topic_haystack(ctx: PolicyContext) -> str:
168         parts = [ctx.source.title or "", ctx.content_item.text or ""]
169         return "\n".join(parts).casefold()
170
171
172     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173         return tuple(topic for topic in topics if topic.casefold() in haystack)
174
175
176     async def evaluate_pipeline(
177         ctx: PolicyContext,
178         stages: list[PolicyStage],
179     ) -> tuple[bool, list[StageResult]]:
180         """Evaluate stages in order and short-circuit on the first failure."""
181         results: list[StageResult] = []
182         for stage in stages:
183             result = await stage.evaluate(ctx)
184             results.append(result)
185             if not result.passed:
186                 return False, results
187         return True, results
188
189
190     __all__ = [
191         "PolicyContext",
192         "PolicyStage",
193         "StageResult",
194         "TopicPreferenceStage",
195         "TopicPreferences",
196         "evaluate_pipeline",
197         "load_topic_preferences",
198         "parse_topic_preferences",
199     ]
200

```

4. user (2026-07-06T21:29:28.175Z)

```

1     """Telethon ingestion: client factory, event handler, and metadata extraction.
2
3     Architecture (see docs/design.md §5):
4
5         NewMessage event
6         -> extract_video_meta(message) # pure, unit-tested
7         -> dedup via db.get_video()
8         -> FilterEngine.evaluate(meta)
9         -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)
10
11     The metadata extraction is deliberately a pure function taking a Telethon
12     ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
13     of logic that's worth unit-testing (everything else is Telethon/async glue).
14     """
15
16     from __future__ import annotations
17
18     import logging
19     import sqlite3
20     from pathlib import Path

```

```

21     from typing import Protocol
22
23     import aiosqlite
24     from telethon import TelegramClient
25     from telethon.events import NewMessage
26     from telethon.tl.types import (
27         Document,
28         DocumentAttributeAnimated,
29         DocumentAttributeVideo,
30         Message,
31         MessageMediaDocument,
32     )
33
34     from tg_video_filter import db
35     from tg_video_filter.commands import ENGINE_KEY
36     from tg_video_filter.delivery import build_link_text, deliver
37     from tg_video_filter.filters import FilterEngine, VideoMeta
38     from tg_video_filter.models import (
39         Channel,
40         ContentItem,
41         DeliveryMode,
42         Source,
43         TelegramSession,
44         User,
45         VideoRecord,
46     )
47     from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage
48
49     log = logging.getLogger(__name__)
50
51
52     # --- pure metadata extraction -----
53
54
55     def extract_video_meta(message: Message) -> VideoMeta | None:
56         """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
57
58         Returns ``None`` when the message has no video/document media. Returns a
59         ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
60         attributes are missing ? the filter engine decides how to treat gaps.
61
62         Telegram stores video metadata in two places:
63         - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
64         - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
65           ``w`` (width), ``h`` (height)
66
67         Note: we inspect ``message.media`` directly rather than the ``.video``
68         convenience property, because the latter only exists on Telethon's *custom*
69         Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
70         used in tests. A Document is considered a video iff it contains a
71         ``DocumentAttributeVideo``.
72         """
73         media = getattr(message, "media", None)
74
75         # Real Telegram messages wrap documents in MessageMediaDocument.
76         # Unwrap it to get the actual Document.
77         if isinstance(media, MessageMediaDocument):
78             media = media.document
79
80         if not isinstance(media, Document):
81             return None
82
83         # Only treat this as a video if the document carries a video attribute.
84         has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in
media.attributes)

```

```

85         if not has_video_attr:
86             return None
87
88         meta = VideoMeta(
89             size_bytes=media.size,
90             mime_type=media.mime_type or None,
91         )
92
93         # Find the video attribute for duration/dimensions and round/nosound flags.
94         video_update: dict = {}
95         for attr in media.attributes:
96             if isinstance(attr, DocumentAttributeVideo):
97                 if attr.duration:
98                     video_update["duration_s"] = int(round(attr.duration))
99                 if attr.w:
100                     video_update["width"] = attr.w
101                 if attr.h:
102                     video_update["height"] = attr.h
103                 video_update["is_round"] = attr.round_message # may be None
104                 video_update["nosound"] = attr.nosound # may be None
105                 break
106
107         # Check for GIF (DocumentAttributeAnimated present).
108         is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)
109
110         meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
111         return meta
112
113
114 # --- client factory -----
115
116
117 def build_client(
118     session: TelegramSession, settings: SettingsLike
119 ) -> TelegramClient: # pragma: no cover
120     """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).
121
122     The session path is read from the ``telegram_sessions.session_path`` column
123     (data-driven, no longer derived from the PK at runtime) and resolved
124     against ``settings.sessions_dir``. The directory is created if missing.
125     """
126     sessions_dir = Path(settings.sessions_dir)
127     sessions_dir.mkdir(parents=True, exist_ok=True)
128     # session_path is stored as a basename (e.g. "user_111.session"); join with
129     # the runtime sessions_dir so the migration did not need to know the path.
130     session_path = sessions_dir / session.session_path
131
132     client = TelegramClient(
133         str(session_path),
134         api_id=settings.tg_api_id,
135         api_hash=settings.tg_api_hash,
136     )
137     return client
138
139
140 class SettingsLike(Protocol):
141     """Structural type for the subset of Settings the client factory needs.
142
143     Using a Protocol keeps the module importable in tests without dragging in
144     the real Settings class. At runtime ``config.Settings`` satisfies this shape.
145     """
146
147     tg_api_id: int
148     tg_api_hash: str
149     sessions_dir: str

```

```

150
151
152 # --- event handler wiring -----
153
154
155 def make_new_message_handler(
156     *,
157     user: User,
158     conn: aiosqlite.Connection,
159     engine_holder: dict[str, FilterEngine],
160     client: TelegramClient,
161     delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
162     delivery_target: str = "me",
163 ) -> _NewMessageCallback:
164     """Return an async callback for ``events.NewMessage``.
165
166     The callback:
167     1. extracts VideoMeta + document_id (skips if not a video),
168     2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
169        plus the existing (channel, message) instance dedup,
170     3. runs the filter engine (read from ``engine_holder`` so config
171        changes via ``/set`` take effect immediately),
172     4. inserts a VideoRecord row,
173     5. delivers a match to ``delivery_target`` (see ADR 0005).
174     """
175
176     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
177         message: Message = event.message # type: ignore[assignment]
178         if message is None:
179             return # Telethon edge case: event has no message (shouldn't happen)
180         # Only process genuine channel/group posts and private messages we
181         # receive ? outgoing messages (sent by our own account) are ignored.
182         if getattr(message, "out", False):
183             return
184
185         meta = extract_video_meta(message)
186         if meta is None:
187             return # not a video
188
189         channel_id = _peer_id(message.peer_id)
190         message_id = message.id
191         document_id = _extract_document_id(message)
192
193         # --- watched gate (ADR 0009, Track E) -----
194         # Opt-in: a source is ingested only when watched=True. An absent
195         # sources row means the channel has never been seen ? treat it as
196         # unwatched and lazily create a watched=0 row for /channels
197         # discoverability (ADR 0009 lines 37-44, 130-131).
198         if not await db.should_ingest_source(conn, user.id, channel_id):
199             return # skip unwatched / unknown source
200
201         # --- three-tier dedup (cheapest first) ---
202
203         # Tier 0: same (user, channel, message) instance ? already processed.
204         existing = await db.get_video(conn, user.id, channel_id, message_id)
205         if existing is not None:
206             return
207
208         # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
209         if document_id is not None:
210             existing = await db.get_video_by_document_id(conn, user.id, document_id)
211             if existing is not None:
212                 log.info(
213                     "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
214                     "original_channel=%s original_msg=%s",

```



```

215         user.id,
216         document_id,
217         channel_id,
218         message_id,
219         existing.channel_id,
220         existing.message_id,
221     )
222     return
223
224     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
225     if (
226         meta.size_bytes is not None
227         and meta.duration_s is not None
228         and meta.height is not None
229         and meta.width is not None
230     ):
231         existing = await db.get_video_by_content_fingerprint(
232             conn,
233             user.id,
234             meta.size_bytes,
235             meta.duration_s,
236             meta.height,
237             meta.width,
238         )
239         if existing is not None:
240             log.info(
241                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
242                 "channel=%s msg=%s original_channel=%s original_msg=%s",
243                 user.id,
244                 meta.size_bytes,
245                 meta.duration_s,
246                 meta.height,
247                 meta.width,
248                 channel_id,
249                 message_id,
250                 existing.channel_id,
251                 existing.message_id,
252             )
253             return
254
255     source = await db.get_source_by_provider_id(conn, user.id, "telegram",
256 str(channel_id))
257
258     # Lazily record the channel (best-effort; failures are non-fatal).
259     try:
260         channel = await db.upsert_channel(
261             conn,
262             Channel(user_id=user.id, channel_id=channel_id,
263 title=_channel_title(event)),
264         )
265         if channel.id is not None:
266             source = await db.get_source_by_id(conn, channel.id)
267     except Exception:
268         log.warning(
269             "failed to upsert channel %s for user %s", channel_id, user.id,
270 exc_info=True
271         )
272
273     text = _message_text(message)
274     result = engine_holder[ENGINE_KEY].evaluate(meta)
275     matched = result.matched
276     reasons = list(result.reasons)
277     if matched and source is not None:
278         topic_result = await _evaluate_topic_preferences(
279             conn,

```

```

277         user.id,
278         source,
279         _content_item_preview(
280             user.id,
281             source,
282             message_id,
283             document_id,
284             meta,
285             text,
286             matched,
287         ),
288     )
289     if topic_result is not None:
290         reasons.append(topic_result.reason)
291         matched = topic_result.passed
292
293     record = VideoRecord(
294         user_id=user.id,
295         channel_id=channel_id,
296         message_id=message_id,
297         text=text,
298         document_id=document_id,
299         duration_s=meta.duration_s,
300         width=meta.width,
301         height=meta.height,
302         size_bytes=meta.size_bytes,
303         mime_type=meta.mime_type,
304         matched=matched,
305     )
306     try:
307         inserted = await db.insert_video(conn, record)
308     except sqlite3.IntegrityError:
309         # Lost a dedup race: a concurrent delivery of the same file
310         # (same document_id or same message instance) won the insert.
311         # The three-tier check above already classified this as new, so
312         # the collision means another task is processing (or has
313         # processed) it ? log and skip.
314         log.info(
315             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
316             user.id,
317             channel_id,
318             message_id,
319             document_id,
320         )
321         return
322     log.info(
323         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
324         user.id,
325         channel_id,
326         message_id,
327         matched,
328         reasons,
329     )
330
331     if matched:
332         # Build link text for LINK/BOTH modes (None for FORWARD).
333         link_text = None
334         if delivery_mode != DeliveryMode.FORWARD:
335             channel_title = _channel_title(event)
336             link_text = build_link_text(
337                 message_link=getattr(message, "link", None),
338                 duration_s=meta.duration_s,
339                 height=meta.height,
340                 size_bytes=meta.size_bytes,
341                 channel_title=channel_title,

```

```

342         )
343         await deliver(
344             client,
345             message,
346             conn,
347             inserted.id,
348             delivery_mode,
349             link_text=link_text,
350             target=delivery_target,
351         )
352
353     return on_new_message
354
355
356 # --- helpers -----
357
358
359 def _extract_document_id(message: Message) -> int | None:
360     """Extract Telegram's ``document.id`` from a video message.
361
362     Returns None if the message has no document media or the id is absent.
363     This is the most reliable content-based dedup key: Telegram reuses the
364     same ``document.id`` when a file is forwarded across channels.
365     """
366     media = getattr(message, "media", None)
367     if isinstance(media, MessageMediaDocument):
368         media = media.document
369     if isinstance(media, Document):
370         return getattr(media, "id", None)
371     return None
372
373
374 def _message_text(message: object) -> str | None:
375     """Return caption/message text normalized for storage, or None if empty."""
376     text = getattr(message, "message", None)
377     if text is None:
378         text = getattr(message, "raw_text", None)
379     if not isinstance(text, str):
380         return None
381     text = text.strip()
382     return text or None
383
384
385 def _content_item_preview(
386     account_id: int,
387     source: Source,
388     message_id: int,
389     document_id: int | None,
390     meta: VideoMeta,
391     text: str | None,
392     matched: bool,
393 ) -> ContentItem:
394     """Build the ContentItem shape used by policy stages before insert."""
395     assert source.id is not None
396     return ContentItem(
397         account_id=account_id,
398         source_id=source.id,
399         provider=source.provider,
400         provider_message_id=str(message_id),
401         content_type="video",
402         text=text,
403         document_id=str(document_id) if document_id is not None else None,
404         duration_s=meta.duration_s,
405         width=meta.width,
406         height=meta.height,

```

```

407         size_bytes=meta.size_bytes,
408         mime_type=meta.mime_type,
409         dedup_key=db.compute_dedup_key(
410             provider=source.provider,
411             document_id=str(document_id) if document_id is not None else None,
412             source_id=source.id,
413             provider_message_id=str(message_id),
414             size_bytes=meta.size_bytes,
415             duration_s=meta.duration_s,
416             height=meta.height,
417             width=meta.width,
418         ),
419         matched=matched,
420     )
421
422
423     async def _evaluate_topic_preferences(
424         conn:aiosqlite.Connection,
425         account_id: int,
426         source: Source,
427         content_item: ContentItem,
428     ) -> StageResult | None:
429         """Evaluate ADR 0012 topic preferences for a candidate item."""
430         policy = await db.get_default_policy(conn, account_id)
431         if policy is None:
432             return None
433         return await TopicPreferenceStage().evaluate(
434             PolicyContext(content_item=content_item, source=source, policy=policy)
435         )
436
437
438     def _peer_id(peer: object) -> int:
439         """Best-effort numeric id for the peer of a message.
440
441         Telegram represents channel/group/user peers as different TL types; we
442         collapse them to a single int for dedup/storage. The exact subclass isn't
443         important ? only that the id is stable per chat.
444
445         # Each Peer* type has a field like channel_id / chat_id / user_id.
446         for field in ("channel_id", "chat_id", "user_id"):
447             val = getattr(peer, field, None)
448             if val is not None:
449                 return int(val)
450         # Fallback: hash the repr so we never collide with real ids (negative).
451         return -abs(hash(repr(peer)) % (2**31))
452
453
454     def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
455         """Best-effort channel title from the event, if available."""
456         chat = getattr(event, "chat", None)
457         return getattr(chat, "title", None) if chat is not None else None
458
459
460     class _NewMessageCallback:
461         """Structural type annotation for the handler returned by make*_handler."""
462
463         async def __call__(self, event: NewMessage.Event) -> None: ...
464
465
466     __all__ = [
467         "build_client",
468         "extract_video_meta",
469         "make_new_message_handler",
470         "_content_item_preview",
471         "_evaluate_topic_preferences",

```

```
472         "_message_text",
473     ]
474
```

5. assistant (2026-07-06T21:29:30.498Z)

Now let me verify the claims. Let me check `save\_policy\_config` and `get\_default\_policy` to confirm no write-time validation exists, then run a repro.

6. user (2026-07-06T21:29:32.052Z)

```
1284:async def get_default_policy(conn: aiomysqlite.Connection, account_id: int) -> Policy | None:
1285-    """Return the account's default policy (`is_default=1`)."""
1286-    conn.row_factory = aiomysqlite.Row
1287-    async with conn.execute(
1288-        "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1289-        (account_id,),
1290-    ) as cur:
1291-        row = await cur.fetchone()
1292-    return _row_to_policy(row) if row else None
1293-
1294-
1295-def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1296-    """Decode a policy config JSON object.
1297-
1298-    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1299-    expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1300-    nested stage config can coexist.
1301-    """
1302-    decoded = json.loads(raw)
1303-    if not isinstance(decoded, dict):
1304-        raise ValueError("policy config_json must decode to a JSON object")
1305-
1306-
1307-
1308-
1309-
1310-
1311-
1312-
1313-
1314-
1315-
1316-
1317-
1318-
1319-
1320-
1321-
1322-
1323-
1324-
1325-
1326:async def save_policy_config(
1327-    conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
1328-) -> None:
1329-    """Persist the raw default policy config object."""
1330-    existing = await get_default_policy(conn, account_id)
1331-    await upsert_policy(
1332-        conn,
1333-        Policy(
1334-            id=existing.id if existing is not None else None,
1335-            account_id=account_id,
1336-            name=existing.name if existing is not None else "default",
1337-            config_json=_policy_config_dict_to_json(config),
1338-            is_default=True,
1339-            created_at=(existing.created_at if existing is not None else _now()),
1340-            updated_at=_now(),
1341-        ),
1342-    )
1343-
1344-
1345-# --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) ---
1346-
```

7. assistant (2026-07-06T21:29:35.284Z)

`save\_policy\_config` does no topic-schema validation. Now let me run a repro to confirm the fail-closed behavior end-to-end through the wiring path (the stage).

8. user (2026-07-06T21:29:43.499Z)

```
valid-match: passed=True reason='matched include topic: ai'
include-string: passed=False reason='invalid topic config: topic include/exclude values must be
```

```
arrays'
include-ints: passed=False reason='invalid topic config: topic include/exclude arrays must
contain strings'
bad-mode: passed=False reason="invalid topic config: topics.mode must be 'any' or 'all'"
array-config: passed=False reason='invalid topic config: policy config_json must decode to a
JSON object'
bad-json: passed=False reason='invalid topic config: Expecting property name enclosed in double
quotes: line 1 column 2 (char 1)'
```

9. assistant (2026-07-06T21:29:48.870Z)

Confirmed at the stage level. Now the critical question for severity: is this actually reachable/latent, or does F1's `/set` path already validate the config at write time (which would refute the "first bad config edit blackholes delivery" scenario)? Let me check what writes `config\_json` and whether the flat FilterConfig keys coexist. The finding admits "no command able to set topics safely-validated" ? let me verify F1's merge path and whether a `/set` of a normal filter could ever produce a malformed `topics` block.

10. user (2026-07-06T21:29:50.509Z)

```
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:87: config_json TEXT NOT NULL
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:136: config_json TEXT NOT NULL,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:480: config_json TEXT
NOT NULL
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:483: "user_id,
config_json",
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:626: INSERT OR IGNORE INTO
policies (account_id, name, config_json, is_default,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:628: SELECT user_id,
'default', config_json, 1,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:699: CASE WHEN
json_valid(config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:700: THEN
json_extract(config_json, '$.delivery_target') END,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:705: CASE WHEN
json_valid(config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:706: THEN
json_extract(config_json, '$.delivery_target') END,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:730: CASE WHEN
json_valid(fcd.config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:731: THEN
json_extract(fcd.config_json, '$.delivery_target') END,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:889: config_json=row["config_json"],
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1257: """"INSERT INTO policies
(account_id, name, config_json, is_default,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1261: config_json=excluded.config_json,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1267: policy.config_json,
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1295: def
_policy_config_json_to_dict(raw: str) -> dict[str, object]:
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1298: ADR 0012 keeps future policy
keys in ``policies.config_json``. The value is
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1304: raise ValueError("policy
config_json must decode to a JSON object")
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1308: def
_policy_config_dict_to_json(config: dict[str, object]) -> str:
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1323: return
_policy_config_json_to_dict(policy.config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1326: async def save_policy_config(
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1337: config_json=_policy_config_dict_to_json(config),
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1761: Invalid ``config_json``
still raises from JSON/Pydantic validation; callers
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1767: return
```

```

FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1773:   ADR 0012: future pipeline
keys live in the same ``policies.config_json``
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1775:   merge the flat config fields
into the existing object so unknown policy
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1782:   raw_config =
_policy_config_json_to_dict(existing.config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1793:   await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\db.py:1862:   "save_policy_config",
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\models.py:191:   future
enrichment/security stages grow into the same ``config_json`` column.
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\models.py:201:   config_json: str = Field(
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\models.py:214:# destinations (backfilled
from filter_config_deprecated.config_json), writes
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\models.py:303:   Stored in the DB as a
JSON blob (``filter_config.config_json``) so adding a
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\policy.py:95:   """Load topic preferences
from ``policy.config_json``."""
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\policy.py:96:   decoded =
json.loads(policy.config_json)
C:\Users\avidu\Projects\wt-pr51\src\tg_video_filter\policy.py:98:   raise
ValueError("policy config_json must decode to a JSON object")

```

11. assistant (2026-07-06T21:29:54.024Z)

Let me look at the F1 merge path (lines 1770-1793) to confirm that the `/set` command (F1) never writes a malformed `topics` block, and that there's genuinely no validated write path for topics in this PR's diff.

12. user (2026-07-06T21:29:55.083Z)

```

1769
1770   async def save_filter_config(conn: aioredis.Connection, user_id: int, cfg:
FilterConfig) -> None:
1771       """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1772
1773       ADR 0012: future pipeline keys live in the same ``policies.config_json``
1774       object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1775       merge the flat config fields into the existing object so unknown policy
1776       keys survive legacy ``/set`` command updates.
1777       """
1778       existing = await get_default_policy(conn, user_id)
1779       raw_config: dict[str, object] = {}
1780       if existing is not None:
1781           try:
1782               raw_config = _policy_config_json_to_dict(existing.config_json)
1783           except (json.JSONDecodeError, ValueError):
1784               log.warning(
1785                   "resetting malformed policy config while saving FilterConfig user=%s "
1786                   "policy_id=%s",
1787                   user_id,
1788                   existing.id,
1789                   exc_info=True,
1790               )
1791       raw_config = {}
1792       raw_config.update(cfg.model_dump(mode="json"))
1793       await save_policy_config(conn, user_id, raw_config)
1794
1795
1796   # --- bulk helpers (used by the manager on startup) -----

```

13. assistant (2026-07-06T21:30:08.903Z)

This confirms the finding's own admission: there is no command in this PR that writes a `topics` block at all. `save\_filter\_config` (the F1 `/set` path) only merges flat `FilterConfig` keys and never touches `topics`. The only way to get a `topics` key into `config\_json` is a manual DB edit or a future command. So the malformed-config scenario is genuinely latent, not triggerable by any shipped command.

Also note: `save\_filter\_config` at line 1783 explicitly handles malformed config by resetting it ? so even legacy `/set` fails open on a bad blob. But that's F1, not F2's diff. The F2 evaluate path (policy.py:125-126 ? telethon_client.py:289-291) is the one that fails closed.

Let me verify the key assertion of the finding: that the fail path produces `matched=False` and silently drops with the only signal being the log line. This is clear from telethon_client.py:289-291 (`matched = topic\_result.passed`) and 322-329 (the log line includes reasons). No user-facing error, no dedicated alert.

The finding is accurate. Let me assess severity precisely:

- The wrong-behavior (fail-closed dropping all matched deliveries) is real and ships in this diff (policy.py:125-126 + telethon_client.py:289-291).
- BUT it is **not** reachable by any command in this PR ? the finding itself admits "the F2 diff makes NO command able to set topics safely-validated (only save_policy_config/manual edit exist)". It requires a manual DB edit or a hypothetical future command.
- The fail-closed choice is a design-posture decision; ADR 0012 doesn't specify fail-open vs fail-closed.
- There IS a visible signal (the reason on the log line), just not distinguished from a legitimate mismatch or surfaced to the user.

This is a latent design-inconsistency / posture issue that will bite when a topics-writing command lands, not a bug triggerable today. That matches **medium** severity as filed. It's not high (no shipped command produces wrong delivery today; requires out-of-band manual edit). It's above low (it's a genuine fail-closed-blackhole risk with a real, if latent, failure mode, and it's inconsistent with F1's own fail-open reset at db.py:1783).

14. user (2026-07-06T21:30:18.282Z)

Structured output provided successfully